Laboratório Funcional · Aula 12

Vinicius Ramon Ferreira · RA 24360978-2 · Turma A
Gerado em 17/10/2026, 13:18:16 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora() não é pura porque depende do relógio do sistema: chamada duas vezes com os mesmos parâmetros (nenhum), devolve resultados diferentes, então falha na primeira condição. poe(lista, x) não é pura porque o append altera a própria lista que veio de fora da função, o que é um efeito colateral e falha na segunda condição.

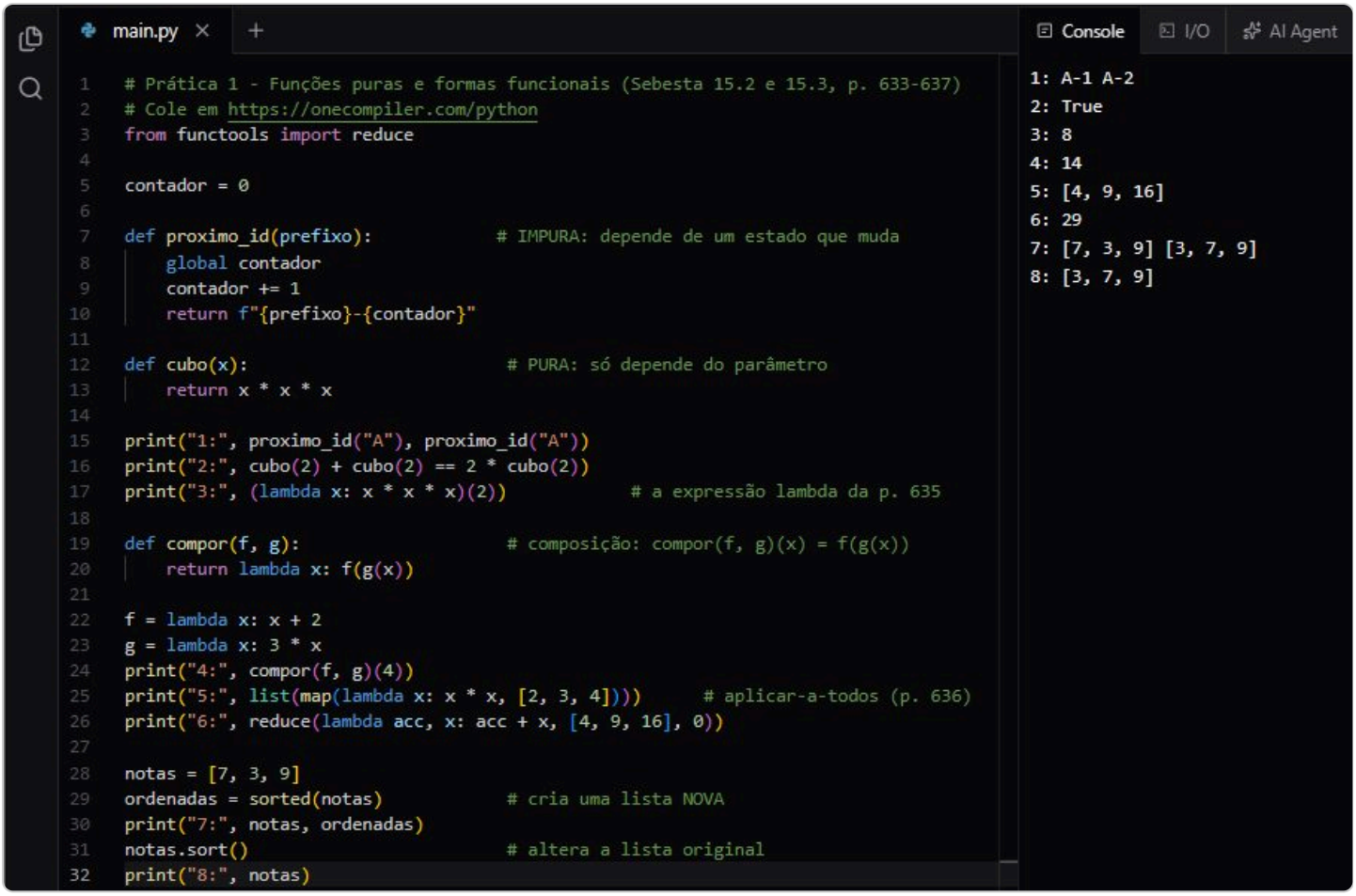
Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Linha 1: 1: A-1 A-2 (o contador aumenta a cada chamada). Linha 4: 4: 14, porque compor(f, g)(4) = f(g(4)) = f(12). Linha 7: 7: [7, 3, 9] [3, 7, 9], porque o sorted cria uma lista nova e a original continua igual.

Saídas (original e com compor(g, f))

```
notas.sort()                        # altera a lista original
print("8:", notas)
148 ms
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```



1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

O proximo_id("A") devolve valores diferentes porque depende da variável global contador, que muda a cada chamada (contador += 1). O resultado não depende só do parâmetro, mas também de quantas vezes a função já foi chamada. Isso dificulta os testes porque não dá para testar a função sozinha: o resultado muda conforme o estado, então é preciso saber ou controlar o contador antes de cada teste (p. 637).

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(3 * 4) = f(12) = 12 + 2 = 14. Já compor(g, f)(4) = g(f(4)) = g(4 + 2) = g(6) = 3 * 6 = 18. A diferença é a ordem: a composição aplica primeiro a função da direita e depois a da esquerda, então trocando a ordem o resultado muda.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma lista nova e deixa a original como estava ([7, 3, 9]). notas.sort() altera a própria lista original (virou [3, 7, 9]). O sorted segue o estilo funcional, porque não tem efeito colateral: não mexe no parâmetro e só devolve um valor.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

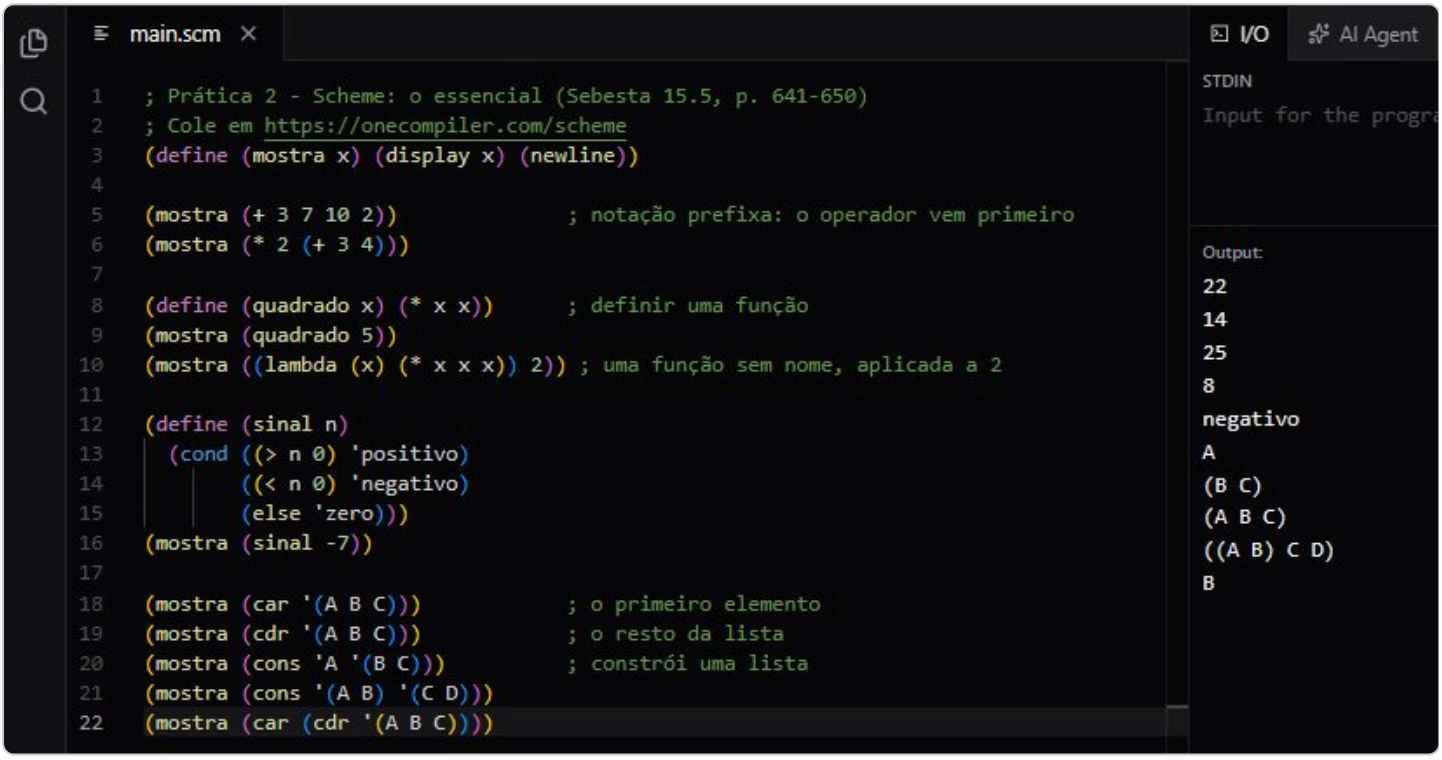
(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```



2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a abreviação de QUOTE e serve para dizer ao Scheme que a lista é um dado, para não ser avaliada. Sem ele, o Scheme tentou avaliar (A B C) como uma chamada de função: usou A como função e B e C como parâmetros. Como A não está definida, deu erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C))))
Resultado: C
(o primeiro cdr tira o A e sobra (B C), o segundo tira o B e sobra (C), e o car pega o C)
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).

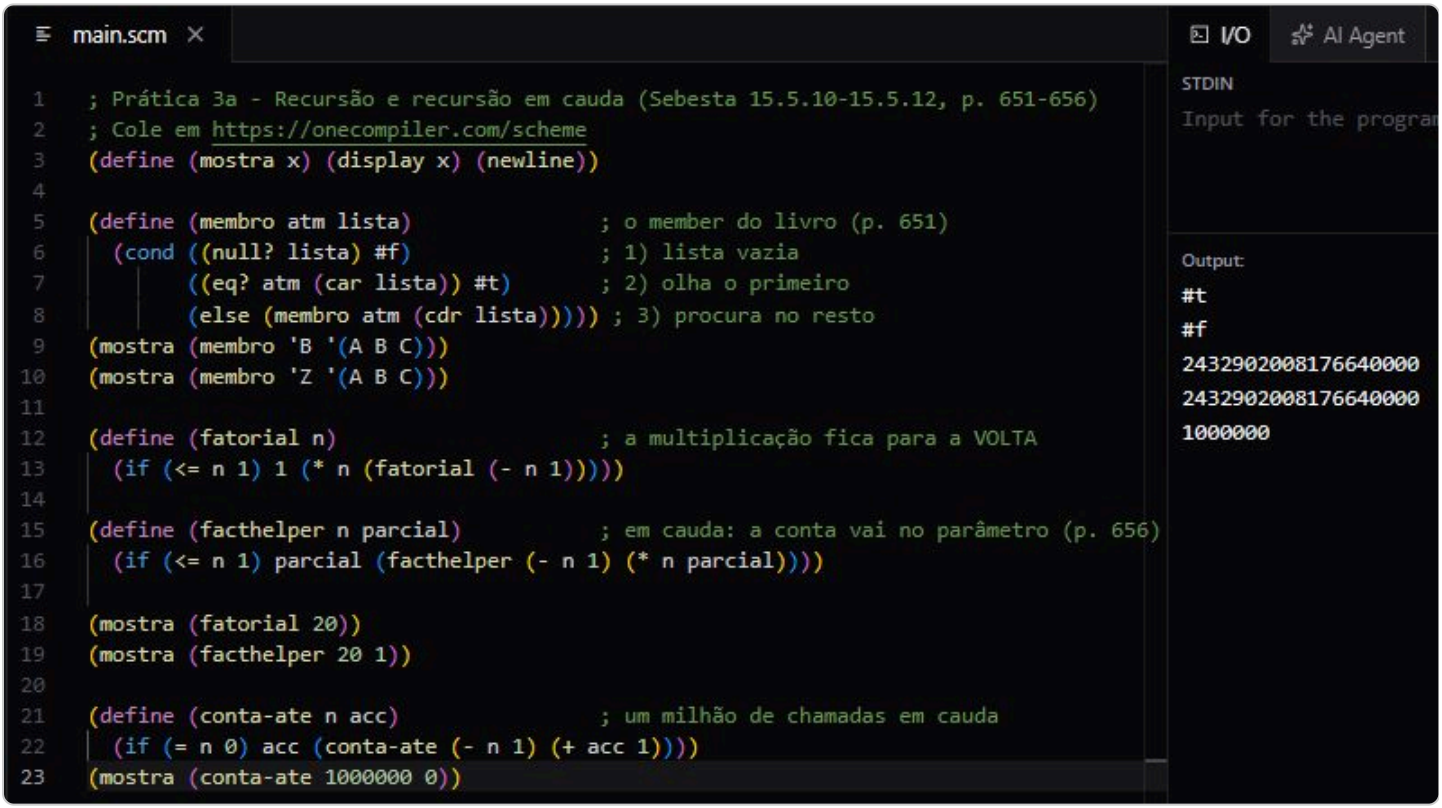
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 5: comum em 10 passos, até 4 multiplicações esperando (expressão de até 36 caracteres); em cauda em 7 passos, nada esperando (até 18 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme acho que vai funcionar, porque a chamada recursiva é a última operação (recursão em cauda) e ele pode reaproveitar o registro de ativação, sem a pilha crescer. No Python acho que não vai funcionar, porque ele não faz essa otimização e deve estourar o limite de chamadas empilhadas.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```



Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
```

```
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a expressão: as multiplicações vão ficando pendentes, esperando a volta da chamada (uma para cada nível), tipo (* 4 (* 3 (* 2 (fatorial 1)))). Na recursão em cauda não cresce, porque a conta é feita antes da chamada e o resultado parcial vai no parâmetro acumulador, então nada fica esperando. Por isso a comum usa mais espaço quanto maior o n, e a de cauda fica constante.

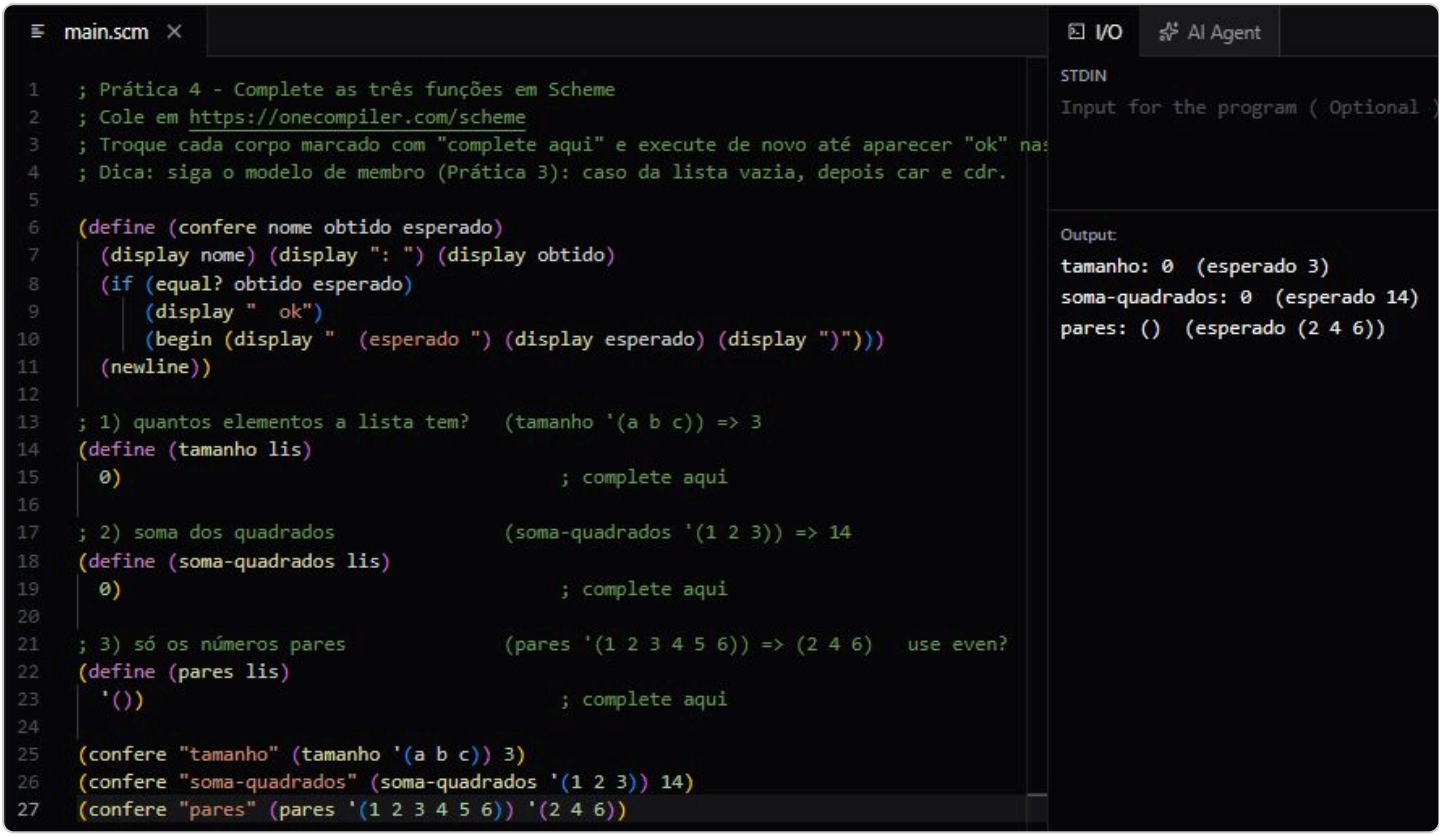
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme faz a otimização da chamada em cauda: como a chamada recursiva é a última operação, ele reaproveita o registro de ativação e a pilha não cresce, então um milhão de chamadas funciona. O Python não faz essa otimização, então cada chamada empilha um novo registro de ativação, e ele ainda limita a profundidade da recursão (por padrão, cerca de 1000). Com um milhão de chamadas estourou o limite e deu RecursionError, mesmo a função sendo recursiva em cauda.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))
```




```
; Prática 4 - Complete as três funções em Scheme
; Cole em https://onecompiler.com/scheme
; Troque cada corpo marcado com "complete aqui" e execute de novo até aparecer "ok" nas
; Dica: siga o modelo de membro (Prática 3): caso da lista vazia, depois car e cdr.

(define (confere nome obtido esperado)
  (display nome) (display ": ") (display obtido)
  (if (equal? obtido esperado)
      (display "  ok")
      (begin (display "  (esperado ") (display esperado) (display ")")))
  (newline))

; 1) quantos elementos a lista tem?   (tamanho '(a b c)) => 3
(define (tamanho lis)
  0)                                    ; complete aqui

; 2) soma dos quadrados               (soma-quadrados '(1 2 3)) => 14
(define (soma-quadrados lis)
  0)                                    ; complete aqui

; 3) só os números pares              (pares '(1 2 3 4 5 6)) => (2 4 6)   use even?
(define (pares lis)
  '())                                  ; complete aqui

(confere "tamanho" (tamanho '(a b c)) 3)
(confere "soma-quadrados" (soma-quadrados '(1 2 3)) 14)
(confere "pares" (pares '(1 2 3 4 5 6)) '(2 4 6))
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi o pares, porque precisa decidir se o elemento entra ou não no resultado. Em todas as funções usei o null? para o caso da lista vazia (devolve 0 ou '()), o car para pegar o primeiro elemento e o cdr para chamar a função de novo com o resto da lista. No tamanho o car nem é usado, só conto 1 e sigo no cdr. No soma-quadrados uso o car para elevar ao quadrado. No pares uso o car no even? e no cons, e se não for par só passo o cdr.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16]. primeiro 42 (div 1 0) imprime 42, sem erro, porque o segundo parâmetro nunca é usado e o Haskell só calcula o que é necessário.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

Main.hs

```haskell
-- Prática 5 - Haskell: padrões, compreensões e avaliação preguiçosa (Sebesta 15.8, p.
-- Cole em https://onecompiler.com/haskell
fact 0 = 1                                  -- casamento de padrões (p. 667)
fact n = n * fact (n - 1)

factors n = [i | i <- [1 .. n `div` 2], n `mod` i == 0]   -- compreensão de lista (p. 6

sort []    = []                             -- quicksort (p. 669, corrigido)
sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]

squares = [n * n | n <- [0 ..]]             -- uma lista INFINITA (p. 670)

member b []    = False                      -- o member do livro (p. 670)
member b (a:x) = (a == b) || member b x

primeiro x _ = x

main :: IO ()
main = do
  print (fact 20)
  print (factors 28)
  print (sort [7, 2, 9, 4, 2, 8])
  print (take 5 squares)
  print (takeWhile (< 50) squares)
  print (member 16 squares)
  print (primeiro 42 (div 1 0))
  -- print (member 17 squares)              -- tire o "--" do começo desta linha
```

I/O AI Agent

STDIN

Input for the program (

Output:

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa: só calcula um valor quando ele é necessário. O take 5 squares só precisa dos 5 primeiros elementos, então a lista infinita só é gerada até aí. Já o primeiro 42 (div 1 0) não deu erro porque o Haskell é não estrito: a função primeiro ignora o segundo parâmetro, então o div 1 0 nunca é calculado.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O member 16 squares terminou porque o 16 está na lista (4 * 4) e a busca para ao achar. O member 17 squares não termina porque o 17 não é um quadrado: ele nunca encontra e a lista é infinita, então continua procurando para sempre. Como squares está em ordem crescente, dá para desistir quando o elemento atual passa do valor buscado:

member2 b []   = False
member2 b (a:x) = (a == b) || (a < b && member2 b x)

Assim, member2 17 squares dá False ao chegar no 25.

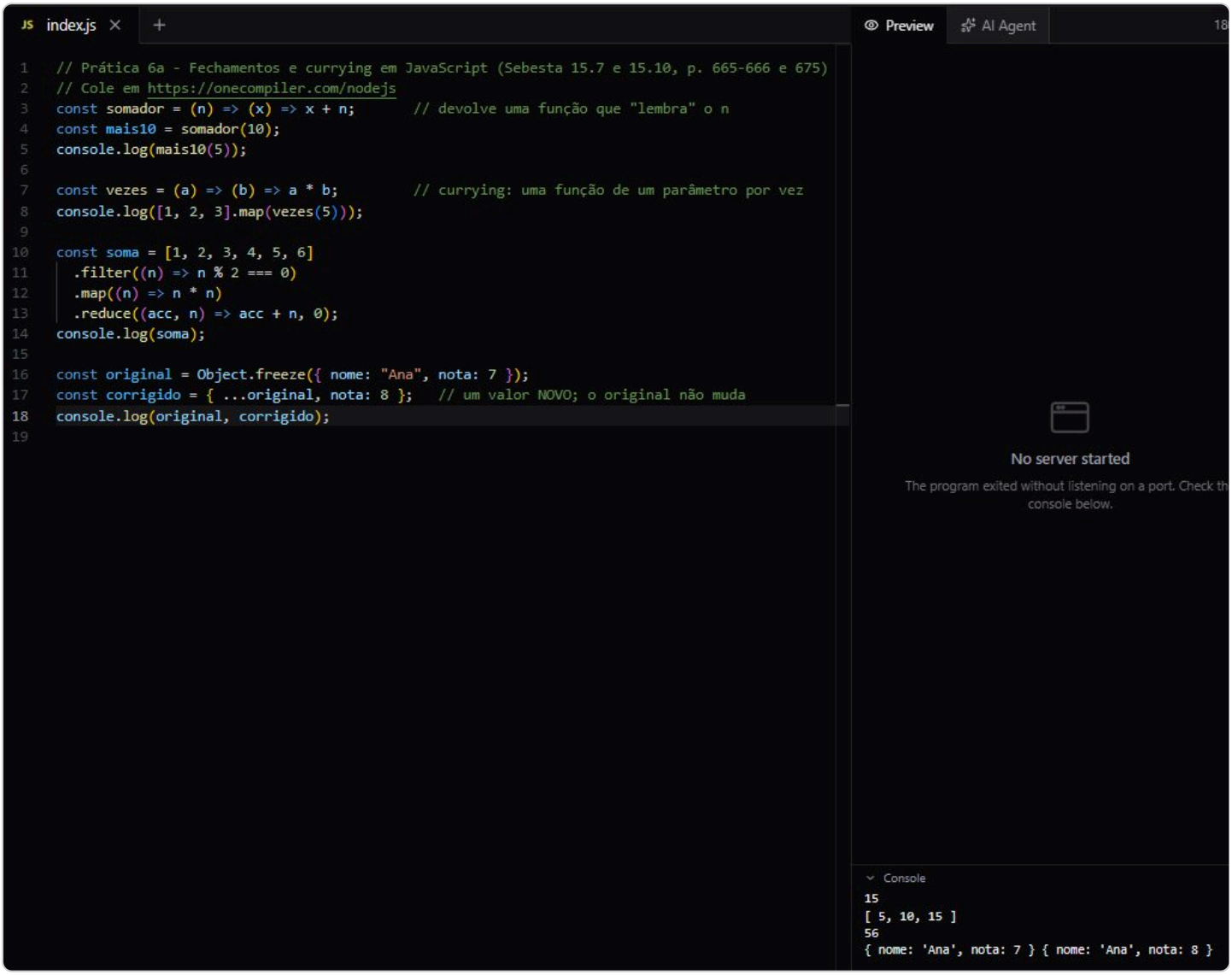
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15, e [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

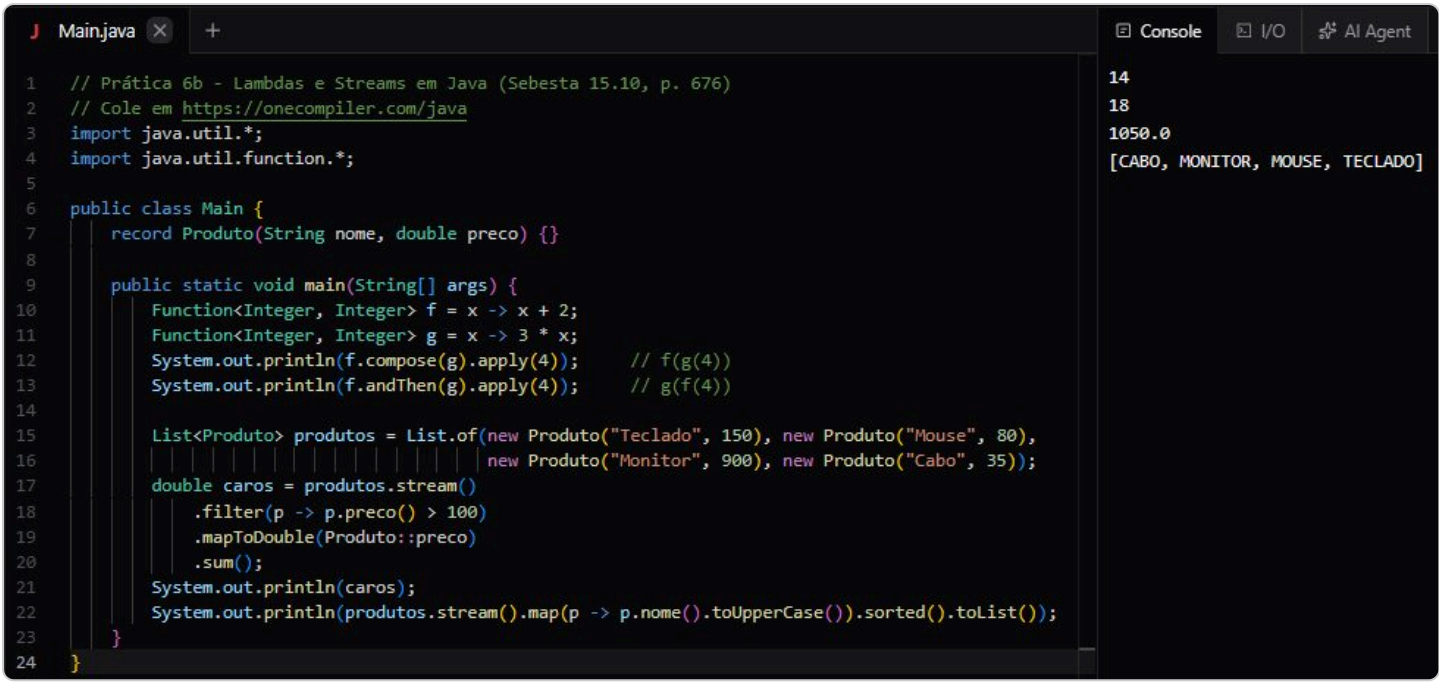
Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```



Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa do fechamento: a função devolvida guarda uma ligação com o ambiente onde foi criada. A variável n (valor 10) é capturada e continua existindo enquanto a função existir, mesmo depois de somador(10) ter terminado.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Porque a ordem é diferente. f.compose(g) aplica primeiro g e depois f, ou seja, f(g(4)) = f(12) = 14. Já f.andThen(g) aplica primeiro f e depois g, ou seja, g(f(4)) = g(6) = 18. O f.compose(g) corresponde ao compor(f, g) da Prática 1.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

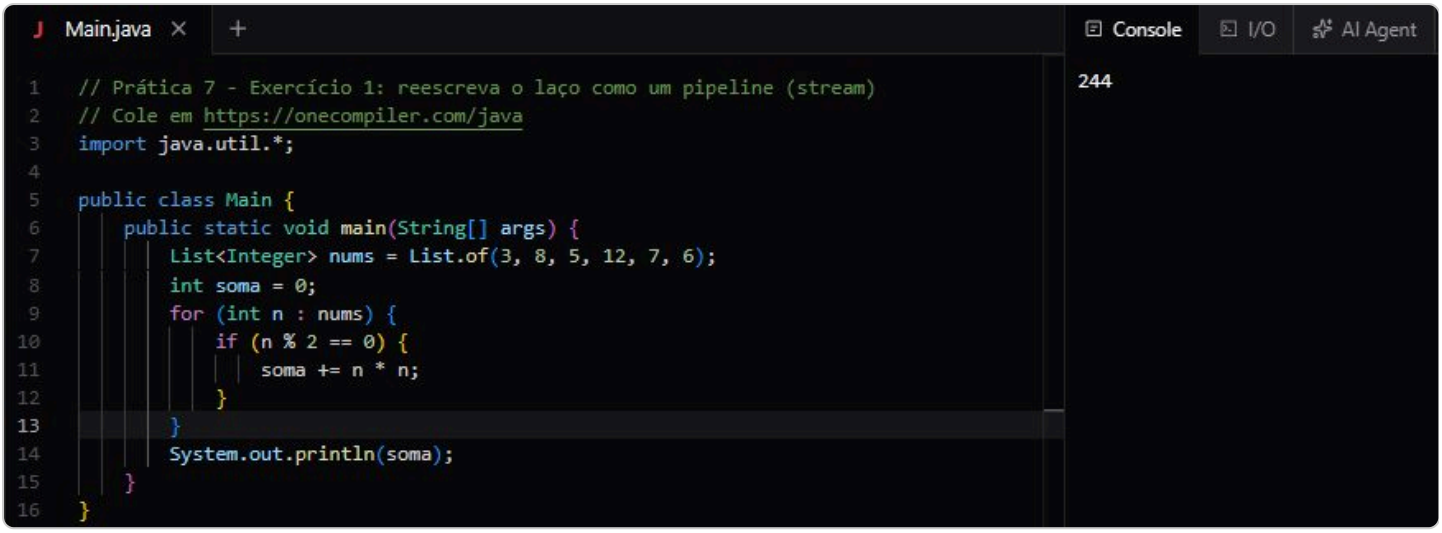
```
int soma = nums.stream()
    .filter(n → n % 2 == 0)
    .mapToInt(n → n * n)
    .sum();
System.out.println(soma);
```

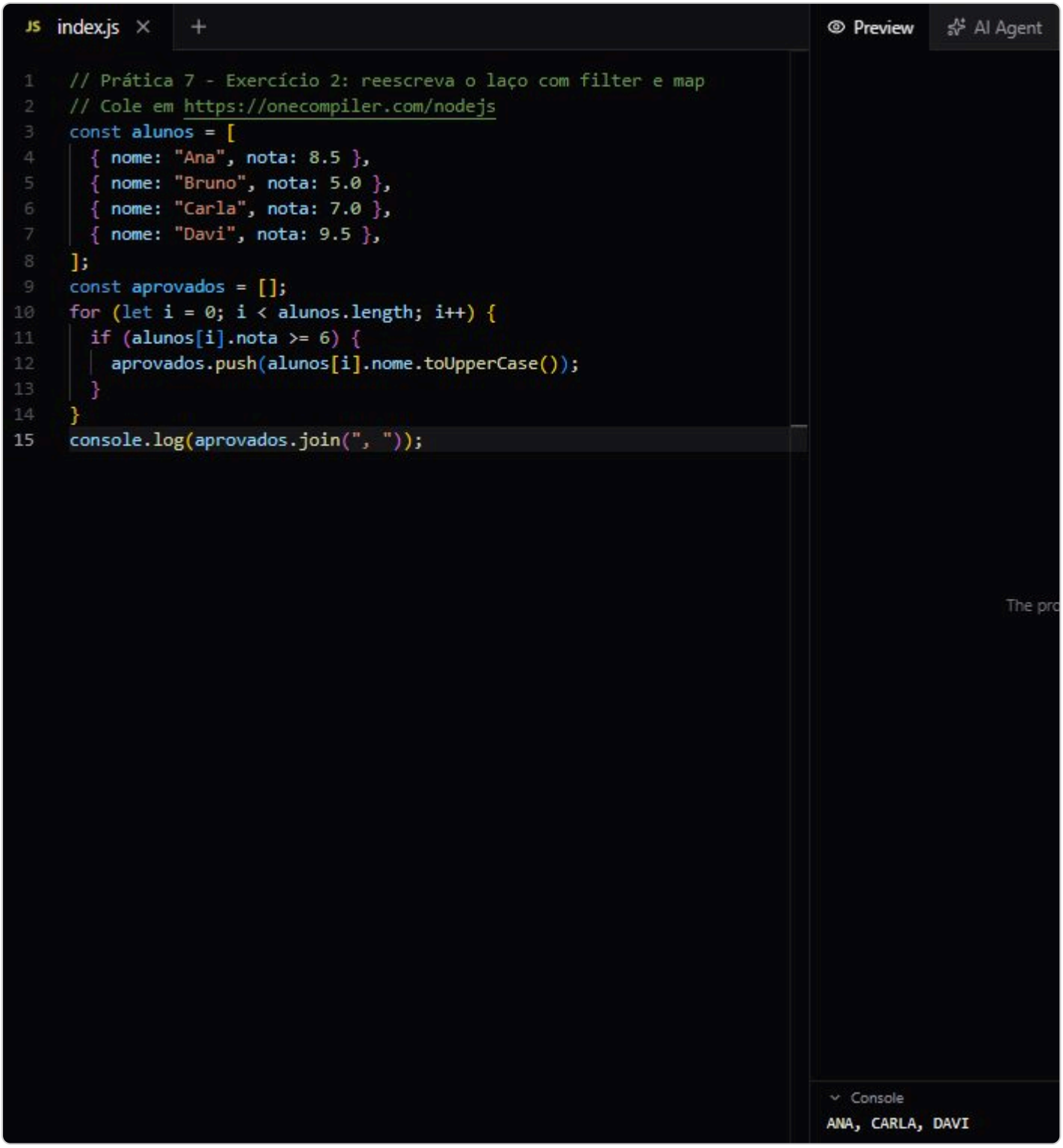
7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const aprovados = alunos
  .filter((a) ⇒ a.nota ⩾ 6)
  .map((a) ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
```


Main.java
Console
I/O
AI Agent
244
// Prática 7 - Exercício 1: reescreva o laço como um pipeline (stream)
// Cole em https://onecompiler.com/java
import java.util.*;
public class Main {
public static void main(String[] args) {
List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
int soma = 0;
for (int n : nums) {
if (n % 2 == 0) {
soma += n * n;
}
}
System.out.println(soma);
}
}



7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão com laço é mais familiar no começo, mas a funcional diz direto o que está sendo feito (filtra, transforma, soma), sem variável acumuladora, então acho ela mais fácil de ler. Para depurar, o laço é mais fácil, porque dá para colocar um print ou um breakpoint no meio e ver o valor intermediário. No pipeline é mais difícil olhar o meio da cadeia.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender, porque tem uma linha só e diz o que fazer (gera a lista, eleva ao cubo, soma), sem contador nem variável que muda. Aceitaria o custo de 2 vezes mais tempo em sistemas em que o importante é a correção e a facilidade de manter, como sistemas financeiros, compiladores ou processamento de dados. Não aceitaria em sistemas em que o desempenho é crítico, como jogos, sistemas embarcados ou de tempo real.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi o Python dar RecursionError com um milhão de chamadas, mesmo a função sendo recursiva em cauda, enquanto o Scheme resolveu sem problema.
(troque por algo que realmente te surpreendeu, se for outra coisa)

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está na primeira compreensão: está escrito b <- h, mas deveria ser b <= h. Com o <-, o Haskell entende que h é uma lista da qual b é tirado, e não uma comparação de filtro. Isso gera um erro de tipo: como h também é elemento da lista que está sendo ordenada, o compilador chega a um tipo infinito (algo como "Occurs check: cannot construct the infinite type: a ~ [a]"). Correção: sort [b | b <- t, b <= h].